

ROOT Token

SMART CONTRACT

Security Audit

Performed on Contracts:

RootToken.sol

MD5 Hash: 9a1d123f095921e73ce6be1c1de631a4

Project Repo: <https://github.com/futureversecom/RootTokenAudit>

Commit Hash: 0c4fa1222746022ff3b0defffd4c68e2460acc4d

Platform

EVM

hashlock.com.au

NOVEMBER 2023

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Standardised Checks	9
Intended Smart Contract Functions	11
Function List	12
Code Quality	13
Audit Resources	13
Dependencies	13
Severity Definitions	14
Audit Findings	14
Centralisation	17
Conclusion	18
Our Methodology	19
Disclaimers	21
About Hashlock	22

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Futureverse team partnered with Hashlock to conduct a security audit of their ERC 20 smart contract for Root token. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that it is ready for mainnet deployment.

Project Context

Futureverse has created Root Token to serve as the centre of the Root Network ecosystem.

Project Name: RootToken

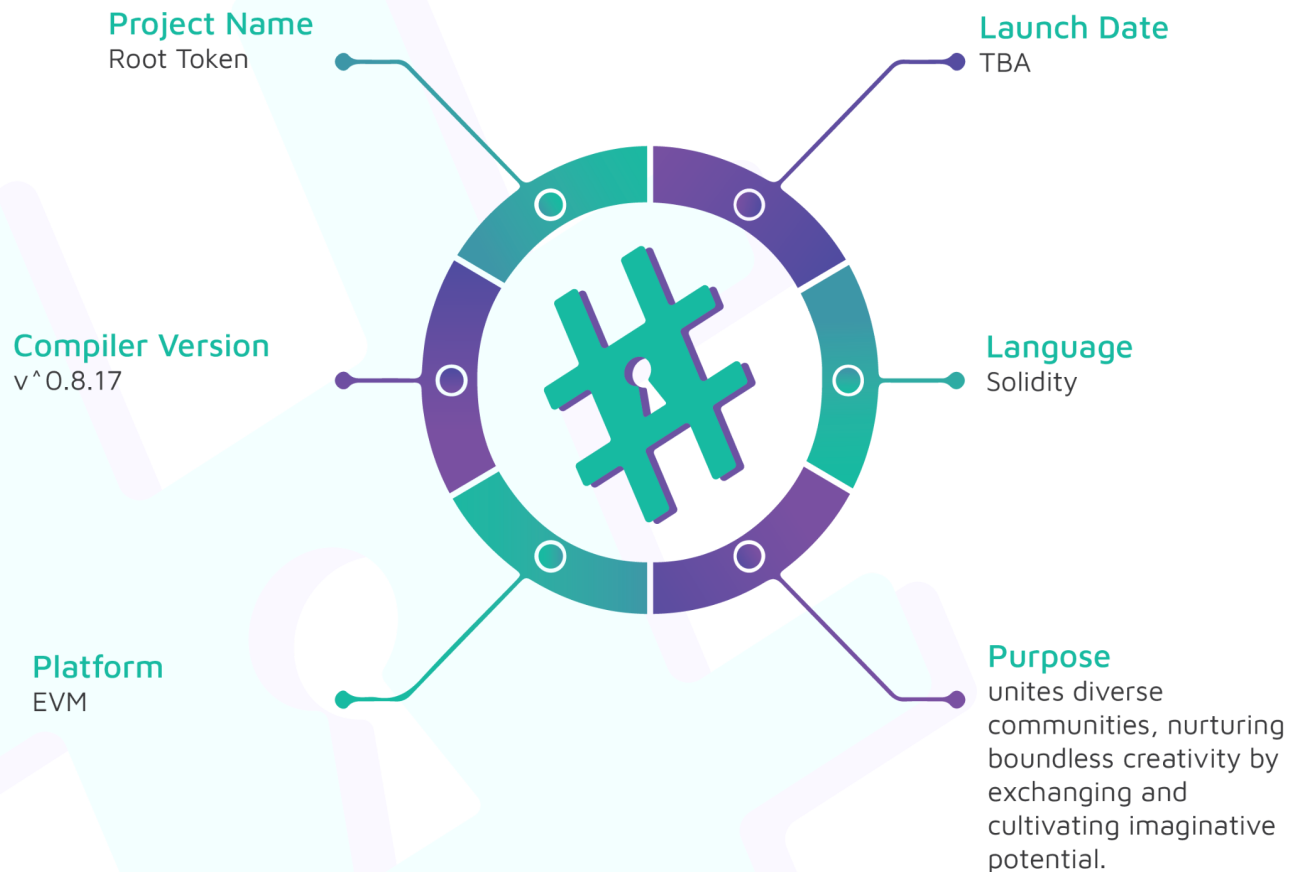
Contract Address: N/A

Compiler Version: ^0.8.17

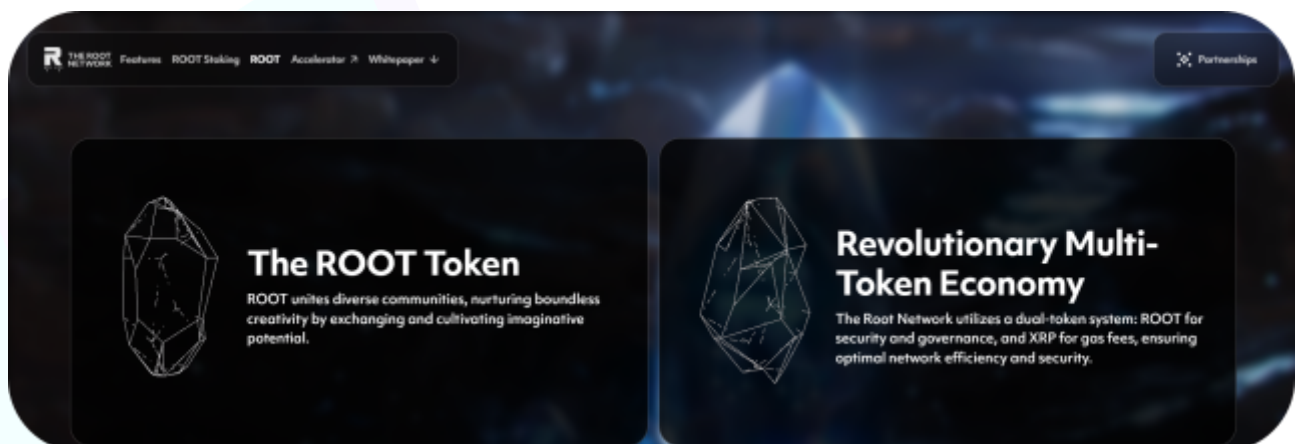
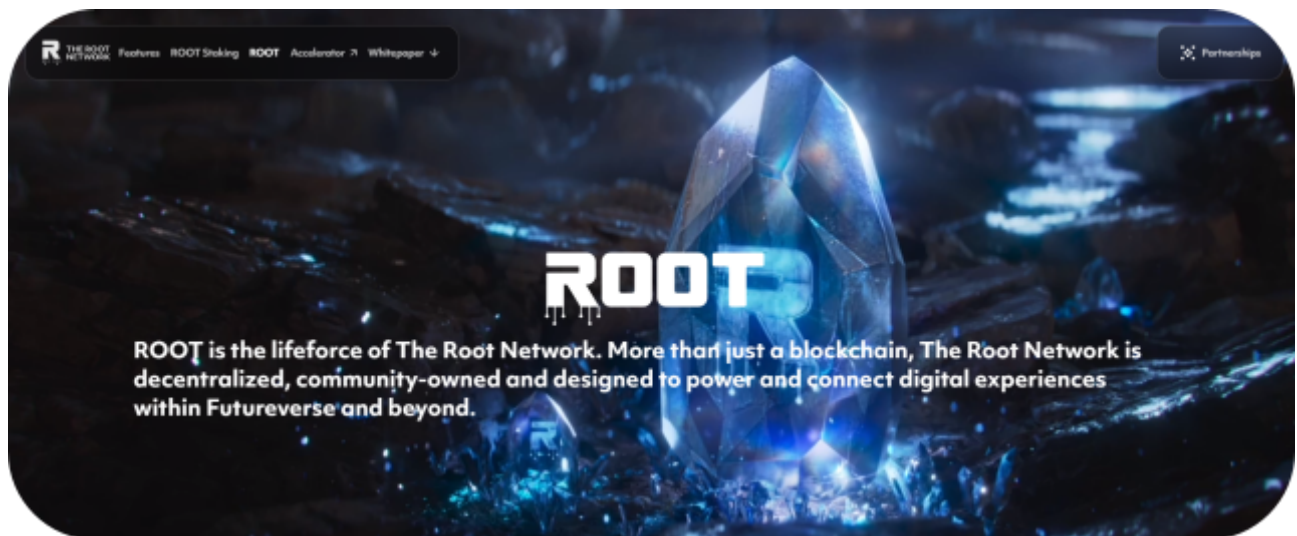
Address: 0xa3d4BEe77B05d4aOC943877558Ce21A763C4fa29

Logo:

ROOT

Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the RootToken Project, the scope of work included a comprehensive review of the smart contract listed below. We tested this smart contract to check for its security and efficiency. These tests were undertaken primarily through manual line by line analysis and were supported by software assisted testing.

Description	Project Review and Security Analysis Report for RootToken Protocol Smart Contracts.
Platform	SKALE
Audit Date	July, 2023
Contract 1	RootToken.sol
Contract 1 MD5 Hash	9a1d123f095921e73ce6be1c1de631a4
Commit Hash	0c4fa1222746022ff3b0defffd4c68e2460acc4d
Revised Contract address	0xa3d4BEe77B05d4a0C943877558Ce21A763C4fa29

Security Rating

After the resolution of our findings, we found the smart contract to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We have identified a few Low severity vulnerabilities and a High that should be addressed before mainnet launch.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit overview section. General overview is presented in the Function list section and all identified issues can be found in the Audit overview section.

All vulnerabilities initially identified have been resolved.

Hashlock found:

1 High severity vulnerabilities

0 Medium severity vulnerabilities

2 Low severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Standardised Checks

Main Category	Subcategory	Result
General Code Checks	Solidity/compiler version stated	Passed
	Consistent pragma version across each contract	Passed
	Outdated Solidity Version	Passed
	Overflow/underflow	Passed
	Correct checks, effects, interaction order	Passed
	Lack of check on input parameters	Passed
	Function input parameters check bypass	Passed
	Correct Access control	Passed
	Built in emergency features	Passed
	Correct event logs	Passed
	Human/contract checks bypass	Passed
	Random number generation/use vulnerability	Passed
	Fallback function misuse	Passed
	Race condition	Passed
	Logical vulnerability	Passed
	Features claimed	Passed
	delegatecall() vulnerabilities	Passed
	Other programming issues	Reviewed
Code Specification	Correctly declared function visibility	Passed
	Correctly declared variable storage location	Passed
	Use keywords/functions to be deprecated	Passed
	Unused code	Passed
Gas Optimization	"Out of Gas" Issue	Passed
	High consumption 'for/while' loop	Passed

	High consumption 'storage' storage	Passed
	Assert() misuse	Passed
Tokenomics Risk	The maximum limit for mintage not set	Passed
	"Short Address" Attack	Passed
	"Double Spend" Attack	Passed

Initial Audit Result: **Failed**

Revised Audit Result: **Passed**

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Contract 1 RootToken.sol Low complexity ERC20 Token.	Contract achieves this functionality.

Function List

	Function	Visibility/Modifiers	Conclusion
1	burn(uint128 _amount)	external,onlyRole	Safe
2	mint(address to, uint128 _amount)	external,onlyRole	Safe
3	decimals()	external,view	Safe
4	_beforeTokenTransfer()	internal,view	Safe
5	pause()	external,onlyRole	Safe
6	unpause()	external,onlyRole	Safe
7	receive()	external,payable	[L-01]

Code Quality

This audit scope involves the solidity smart contract for RootToken, as outlined in the Audit Scope section. All contracts, libraries and interfaces intend to follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation.

The code is extremely well commented and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the RootToken Protocol's smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is fairly straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic after some manual testing. The comments are helpful in understanding the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High severity vulnerabilities can result in material loss of funds, asset loss, access denial, theft of value, and other critical issues that will result in the direct loss of funds controlled by the owners or the community.
Medium	Medium level vulnerabilities should be solved before deployment and require certain edge cases to be met in order for funds to be lost, stolen or inaccessible.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future, but are less important than high and medium level vulnerabilities.
Gas	Gas Optimisations, issues and inefficiencies are addressed to save on gas fees when interacting with the protocol.

Audit Findings

Auditors' Note: The Hashlock team has identified that there is a possibility of a 'Denial of Service' attack in the `Bridge._verifyMessage` function if there are too many validators in the Root Network that need to have their signatures verified.

High

[H-01] Root token uses 1e18 for total supply but decimals is 1e6

Description

The total supply and supply cap for the Root token is intended to be 12 billion tokens. However, it is actually 12 sextillion (1.2×10^{22}).

Vulnerability Details

The `TOTAL_SUPPLY` constant is assigned as `12_000_000_000_000_000_000_000_000_000` inside `RootToken`. Since the `RootToken` has 6 decimals, this is equivalent to 12 sextillion tokens, which is inconsistent with the comment indicating that the token is intended to have a total supply of 12 billion tokens.

Impact

The total supply and supply cap of the Root token is a lot larger than intended.

Recommendation

The `TOTAL_SUPPLY` constant should be assigned as `12_000_000_000_000_000` instead of `12_000_000_000_000_000_000_000_000_000`.

Status

Resolved

Medium

No medium vulnerabilities have been identified.

Low

[L-01] RootToken - receive function is unnecessary

Description

The `RootToken` contract has a payable `receive()` function which reverts. Since it reverts and does not perform any actions, it is unnecessary to include a `receive()` function.

Recommendation

Remove the `receive()` function.

Status

Resolved

[L-02] RootToken - Use scientific notation for large numbers

Description

The RootToken's `TOTAL_SUPPLY` constant is a very large number expressed in standard decimal notation. This is confusing and reduces code readability.

Recommendation

Use scientific notation to express large numbers.

Since `TOTAL_SUPPLY` is incorrect as explained in [\[H-01\]](#), the following line should be the new `TOTAL_SUPPLY` constant:

```
uint128 constant TOTAL_SUPPLY = 12e15;
```

Status

Resolved

Gas

No gas issues were found.

Centralisation

The project values security and utility over decentralisation, but still inherits aspects of decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

At time of audit, the Root Network is still in alpha phase and has a small controlled set of validators, of which most are controlled internally by the project team. However, there are plans to open validation to the public in the distant future.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the RootToken project seems to have a sound and well tested code base, after the resolution of our findings. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

#Hashlock.

